

数据结构与算法 H2 实验报告

1900011604 张植竣 指导老师: 陈斌

北京大学物理学院天文学系

2020 年 3 月 4 日

1 H2.1 验证 `list` 按索引取值的时间复杂度为 $O(1)$

代码

```
# -*- coding: utf-8 -*-
```

```
#[H2.1] 做计时实验, 验证list的按索引取值确实是O(1)
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from timeit import Timer
```

```
#计时实验程序
```

```
def test1():
```

```
    return l[n-1] #list按索引取值函数
```

```
t1=Timer("test1()", "from __main__ import test1")
```

```
x_list1 = []
```

```
y_list1 = []
```

```
for n in range(10000,2510000,10000): #250个数据点
```

```
    l=list(range(n))
```

```
    x_list1.append(n)
```

```
    y_list1.append(t1.timeit(number=1))
```

```
#线性拟合数据点
```

```
def linefit(x, y):
```

```
    N = len(x)
```

```
    sx,sy,sxx,syy,sxy=0,0,0,0,0
```

```
    for i in range(0,N):
```

```
        sx += x[i]
```

```
        sy += y[i]
```

```
        sxx += x[i]*x[i]
```

```
        syy += y[i]*y[i]
```

```
        sxy += x[i]*y[i]
```

```
    k = (sy*sx/N-sxy)/(sx*sx/N-sxx)
```

```
    b = (sy-k*sx)/N
```

```
    R = (sxy-sx*sy/N)/((sxx-sx*sx/N)*(syy-sy*sy/N))**0.5
```

```
    return [k,b,R]
```

```
for i in range(len(x_list1)):
```

```

    x_list1[i]/=1000000
r=linefit(x_list1,y_list1)
print('k='+str(r[0]))
print('b='+str(r[1]))
print('r='+str(r[2]))
print('\nt=%.2fn+%.2f\n'%(r[0], r[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)

#绘图
xmax1 = x_list1[-1]
#x轴的上界
dx1 = x_list1[-1] / 5
#x轴的单位长度
ymax1 = float("{0:.2f}".format(max(y_list1)))
#y轴的上界
dy1 = ymax1 / 5
#y轴的单位长度
x1 = np.arange(0, xmax1 + 0.5*dx1, 0.01) #绘直线用
plt.figure(figsize=(16, 10))
plt.scatter(x_list1, y_list1, color='g', marker='+')
plt.plot(x1, r[0]*x1+r[1], color='r')
plt.title('H2.1', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(s)", fontsize=16)
plt.xlim((0, xmax1 + dx1))
plt.ylim((0, ymax1 + dy1))
xaxis = np.arange(0, xmax1 + dx1, dx1)
yaxis = np.arange(0, 2 * ymax1 + 4 * dy1, dy1)
#使数据点落在图像的中央
plt.xticks(xaxis)
plt.yticks(yaxis)
plt.show()

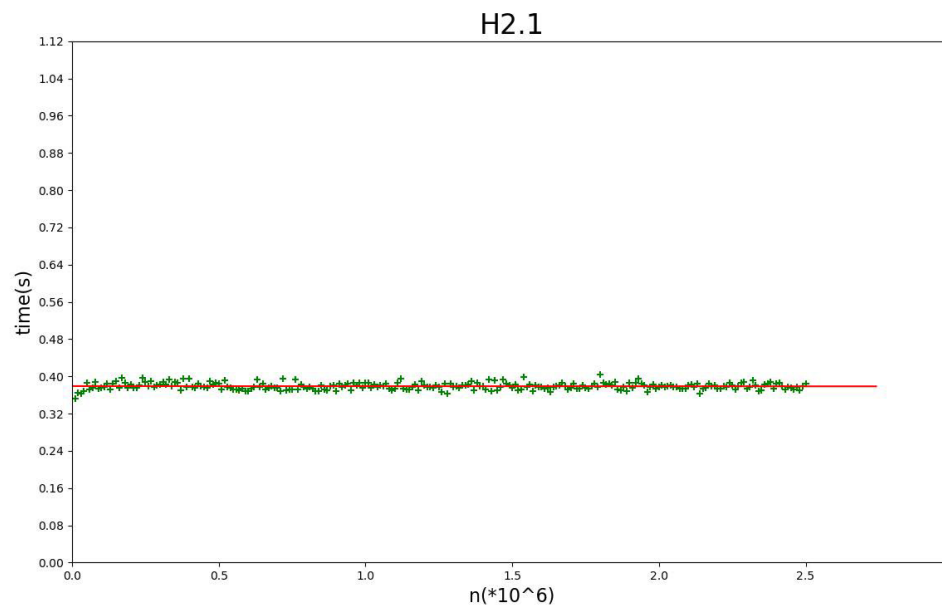
```

数据说明 问题的规模 n ，即列表 l 的长度，范围从 1×10^5 到 2.50×10^7 ，间隔为 1×10^5 ，每一条按索引取值的程序指令复执行 10^6 次，共得到 250 个数据点。

运行结果说明

- 斜率: $k = -0.0002197207315316944$
- 截距: $b = 0.37907678111807197$
- 相关系数: $r = -0.021488761847494306$
- 时间 $t(s)$ 与问题规模 $n(\times 10^6)$ 的关系: $t = 0.00n + 0.38$

实验结果分析 可以看到，运行 `list` 按索引取值函数 10^6 次时，数据点集中分布在水平线 $t = 0.38\text{s}$ 附近，拟合出的斜率 k 几乎为 0（考虑到复运行了 10^6 次，实际上 $k \approx -2.2 \times 10^{-10}$ ），说明 `list` 取值所需时间与问题的规模（即列表长度 n ）关，时间复杂度为 $O(1)$ 。



2 H2.2 验证 dict 中 get item 和 set item 的时间复杂度为 $O(1)$

2.1 H2.2.1 验证 dict 中 get item 的时间复杂度为 $O(1)$

代码

```
# -*- coding: utf-8 -*-
```

```
#[H2.2] 做计时实验，验证dict的get item和set item都是O(1)的
```

```
#[H2.2.1] 做计时实验，验证dict的get item是O(1)的
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from timeit import Timer
```

```
#计时实验程序
```

```
def test21():
```

```
    return d[n-1]
```

```
t21=Timer("test21()", "from __main__ import test21")
```

```
x_list21 = []
```

```
y_list21 = []
```

```
for n in range(10000,2510000,10000): #250个数据点
```

```
    d=dict(list((j,j) for j in range(n)))
```

```
    x_list21.append(n)
```

```
    y_list21.append(t21.timeit(number=1000000))
```

```
#线性拟合数据点
```

```
def linefit(x, y):
```

```
    N = len(x)
```

```
    sx,sy,sxx,syy,sxy=0,0,0,0,0
```

```
    for i in range(0,N):
```

```
        sx += x[i]
```

```
        sy += y[i]
```

```
        sxx += x[i]*x[i]
```

```
        syy += y[i]*y[i]
```

```
        sxy += x[i]*y[i]
```

```
    k = (sy*sx/N-sxy)/(sx*sx/N-sxx)
```

```
    b = (sy-k*sx)/N
```

```
    R = (sxy-sx*sy/N)/((sxx-sx*sx/N)*(syy-sy*sy/N))**0.5
```

```
    return [k,b,R]
```

```
for i in range(len(x_list21)):
```

```
    x_list21[i]/=1000000
```

```
r=linefit(x_list21,y_list21)
```

```
print('k='+str(r[0]))
```

```
print('b='+str(r[1]))
```

```
print('r='+str(r[2]))
```

```
print('\nt=%.2fn+%.2f\n'%(r[0], r[1]))
```

```

#拟合结果（斜率k，截距b，相关系数r）

#绘图
xmax21 = x_list21[-1]
#x轴的上界
dx21 = x_list21[-1] / 5
#x轴的单位长度
ymax21 = float("{0:.2f}".format(max(y_list21)))
#y轴的上界
dy21 = ymax21 / 5
#y轴的单位长度
x21 = np.arange(0, xmax21 + 0.5*dx21, 0.01) #绘直线用
plt.figure(figsize=(16, 10))
plt.scatter(x_list21, y_list21, color='g', marker='+')
plt.plot(x21, r[0]*x21+r[1], color='r')
plt.title('H2.2.1', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(s)", fontsize=16)
plt.xlim((0, xmax21 + dx21))
plt.ylim((0, ymax21 + dy21))
xaxis = np.arange(0, xmax21 + dx21, dx21)
yaxis = np.arange(0, 2 * ymax21 + 4 * dy21, dy21)
#使数据点落在图像的中央
plt.xticks(xaxis)
plt.yticks(yaxis)
plt.show()

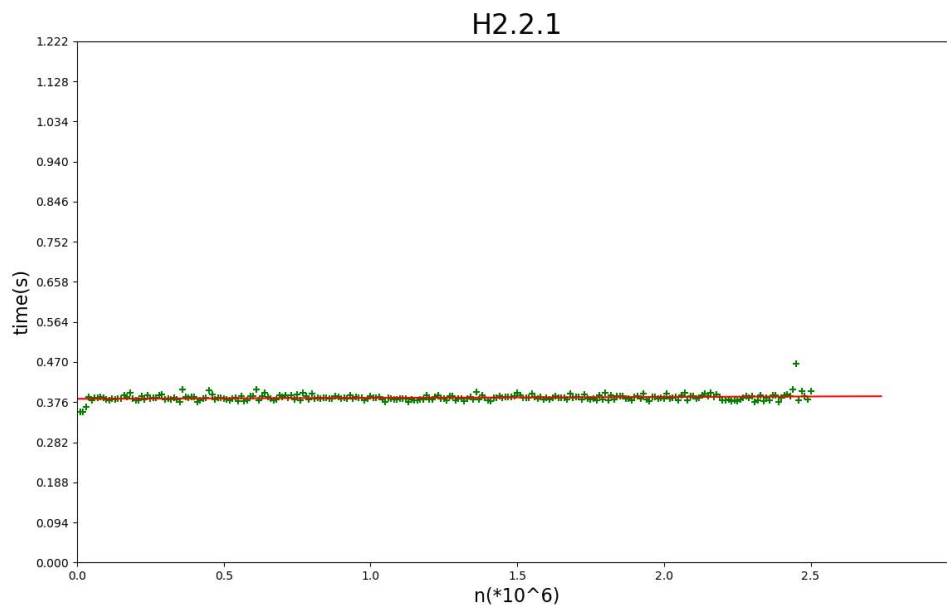
```

数据说明 问题的规模 n ，即字典 d 的长度（键值对数量），范围从 1×10^4 到 2.50×10^6 ，间隔为 1×10^4 ，每一条 `get item` 参数重复执行 10^6 次，共得到 250 个数据点。

运行结果说明

- 斜率: $k = 0.0021657445437530416$
- 截距: $b = 0.3838213509975898$
- 相关系数: $r = 0.19171137995698143$
- 时间 $t(s)$ 与问题规模 $n(\times 10^6)$ 的关系: $t = 0.00n + 0.38$

实验结果分析 可以看到，运行 `dict` 的 `get item` 函数 10^6 次时，数据点集中分布在水平线 $t = 0.38s$ 附近，拟合出的斜率 k 几乎为 0（考虑到重复运行了 10^6 次，实际上 $k \approx 2.2 \times 10^{-9}$ ），说明 `dict` 的 `get item` 函数运行所时间与问题的规模（即字典长度 n ）无关，时间复杂度为 $O(1)$ ，这是由字典本身的性质决定的。



2.2 H2.2.2 验证 *dict* 中 *set item* 的时间复杂度为 $O(1)$

代码

```
# -*- coding: utf-8 -*-
```

```
#[H2.2.2] 做计时实验，验证dict的set item是O(1)的
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from timeit import Timer
```

```
#计时实验程序
```

```
def test22():
```

```
    d[n-1]=n
```

```
t22=Timer("test22()", "from __main__ import test22")
```

```
x_list22 = []
```

```
y_list22 = []
```

```
for n in range(10000,110000,10000): #10个数据点
```

```
    d=dict(list((j,j) for j in range(n)))
```

```
    x_list22.append(n)
```

```
    y_list22.append(t22.timeit(number=1000000))
```

```
    print(n//10000)
```

```
#线性拟合数据点
```

```
def linefit(x, y):
```

```
    N = len(x)
```

```
    sx,sy,sxx,syy,sxy=0,0,0,0,0
```

```
    for i in range(0,N):
```

```
        sx += x[i]
```

```

        sy += y[i]
        sxx += x[i]*x[i]
        syy += y[i]*y[i]
        sxy += x[i]*y[i]
    k = (sy*sx/N-sxy)/(sx*sx/N-sxx)
    b = (sy-k*sx)/N
    R = (sxy-sx*sy/N)/((sxx-sx*sx/N)*(syy-sy*sy/N))**0.5
    return [k,b,R]
for i in range(len(x_list22)):
    x_list22[i]/=1000000
r=linefit(x_list22,y_list22)
print('k='+str(r[0]))
print('b='+str(r[1]))
print('r='+str(r[2]))
print('\nt=%.2fn+%.2fn'%(r[0], r[1]))
#拟合结果（斜率k，截距b，相关系数r）

#绘图
xmax22 = x_list22[-1]
#x轴的上界
dx22 = x_list22[-1] / 5
#x轴的单位长度
ymax22 = float("{0:.2f}".format(max(y_list22)))
#y轴的上界
dy22 = ymax22 / 5
#y轴的单位长度
x22 = np.arange(0, xmax22 + 0.5*dx22, 0.01) #绘直线用
plt.figure(figsize=(16, 10))
plt.scatter(x_list22, y_list22, color='g', marker='+')
plt.plot(x22, r[0]*x22+r[1], color='r')
plt.title('H2.2.2', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(s)", fontsize=16)
plt.xlim((0, xmax22 + dx22))
plt.ylim((0, ymax22 + dy22))
xaxis = np.arange(0, xmax22 + dx22, dx22)
yaxis = np.arange(0, 2 * ymax22 + 4 * dy22, dy22)
#使数据点落在图像的中央
plt.xticks(xaxis)
plt.yticks(yaxis)
plt.show()

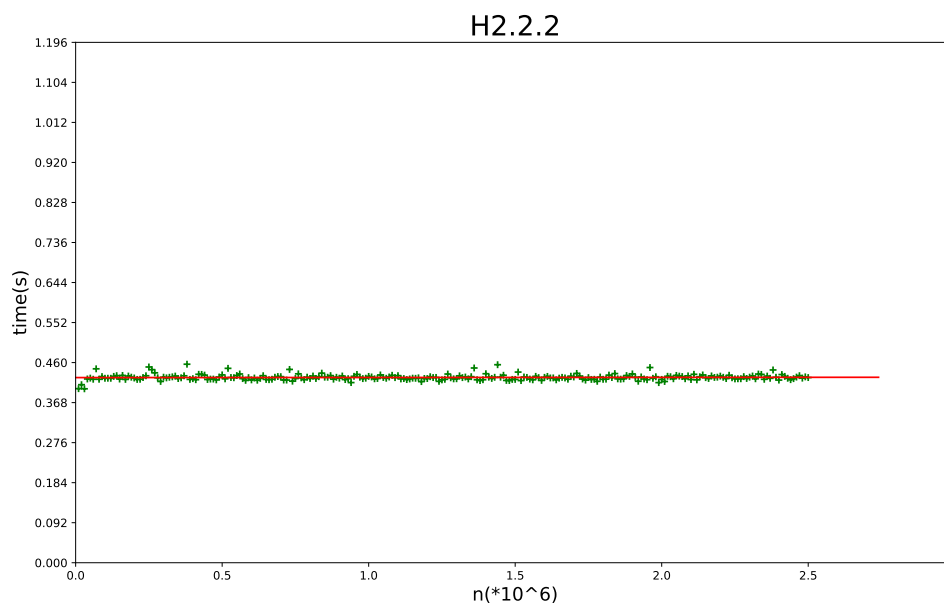
```

数据说明 问题的规模 n ，即字典 d 的长度（键值对数量），范围从 1×10^4 到 2.50×10^6 ，间隔为 1×10^4 ，每一条 `set item` 参数重复执行 10^6 次，共得到 250 个数据点。

运行结果说明

- 斜率: $k = 0.00024256535304588268$
- 截距: $b = 0.42553140448192694$
- 相关系数: $r = 0.026154043941845978$
- 时间 $t(s)$ 与问题规模 $n(\times 10^6)$ 的关系: $t = 0.00n + 0.43$

实验结果分析 可以看到, 运行 `dict` 的 `set item` 函数 10^6 次时, 数据点集中分布在水平线 $t = 0.43s$ 附近, 拟合出的斜率 k 几乎为 0 (考虑到重复运行了 10^6 次, 实际上 $k \approx 2.4 \times 10^{-10}$), 说明 `dict` 的 `set item` 函数运行需时间与问题的规模 (即字典长度 n) 无关, 时间复杂度为 $O(1)$, 这是由字典本身的性质决定的。



3 H2.3 比较 list 和 dict 中 del 操作符的性能

3.1 H2.3.1 list 中 del 操作符的性能

代码

```
# -*- coding: utf-8 -*-
```

```
#[H2.3] 做计时实验，比较list和dict中del操作符的性能
```

```
#H2.3.1 list中del操作符的性能
```

```
import numpy as np
import matplotlib.pyplot as plt
from timeit import Timer

#计时实验程序
def test311():
    del l[0]
t311=Timer("test311()", "from __main__ import test311")
def test312():
    del l[int(n/4)]
t312=Timer("test312()", "from __main__ import test312")
def test313():
    del l[int(n/2)]
t313=Timer("test313()", "from __main__ import test313")
def test314():
    del l[int(3*n/4)]
t314=Timer("test314()", "from __main__ import test314")
def test315():
    del l[-1]
t315=Timer("test315()", "from __main__ import test315")
x_list31 = []
y_list311, y_list312 = [], []
y_list313, y_list314, y_list315 = [], [], []
for n in range(40000,10040000,40000): #250个数据点
    l=list((j,j) for j in range(n))
    x_list31.append(n)
    y_list311.append(t311.timeit(number=10))
    y_list312.append(t312.timeit(number=10))
    y_list313.append(t313.timeit(number=10))
    y_list314.append(t314.timeit(number=10))
    y_list315.append(t315.timeit(number=10))

#线性拟合数据点
def linefit(x, y):
    N = len(x)
    sx,sy,sxx,syy,sxy=0,0,0,0,0
    for i in range(0,N):
```

```

        sx += x[i]
        sy += y[i]
        sxx += x[i]*x[i]
        syy += y[i]*y[i]
        sxy += x[i]*y[i]
    k = (sy*sx/N-sxy)/(sx*sx/N-sxx)
    b = (sy-k*sx)/N
    R = (sxy-sx*sy/N)/((sxx-sx*sx/N)*(syy-sy*sy/N))**0.5
    return [k,b,R]
for i in range(len(x_list31)):
    x_list31[i]/=1000000
    y_list311[i]*=1000
    y_list312[i]*=1000
    y_list313[i]*=1000
    y_list314[i]*=1000
    y_list315[i]*=1000
r1=linefit(x_list31,y_list311)
print('k1='+str(r1[0]))
print('b1='+str(r1[1]))
print('r1='+str(r1[2]))
print('\nt1=%.2fn+%.2f\n'%(r1[0], r1[1]))
#拟合结果（斜率k，截距b，相关系数r）
r2=linefit(x_list31,y_list312)
print('k2='+str(r2[0]))
print('b2='+str(r2[1]))
print('r2='+str(r2[2]))
print('\nt2=%.2fn+%.2f\n'%(r2[0], r2[1]))
#拟合结果（斜率k，截距b，相关系数r）
r3=linefit(x_list31,y_list313)
print('k3='+str(r3[0]))
print('b3='+str(r3[1]))
print('r3='+str(r3[2]))
print('\nt3=%.2fn+%.2f\n'%(r3[0], r3[1]))
#拟合结果（斜率k，截距b，相关系数r）
r4=linefit(x_list31,y_list314)
print('k4='+str(r4[0]))
print('b4='+str(r4[1]))
print('r4='+str(r4[2]))
print('\nt4=%.2fn+%.2f\n'%(r4[0], r4[1]))
#拟合结果（斜率k，截距b，相关系数r）
r5=linefit(x_list31,y_list315)
print('k5='+str(r5[0]))
print('b5='+str(r5[1]))
print('r5='+str(r5[2]))
print('\nt5=%.2fn+%.2f\n'%(r5[0], r5[1]))
#拟合结果（斜率k，截距b，相关系数r）

#绘图

```

```

xmax31 = x_list31[-1]
#x轴的上界
dx31 = (x_list31[-1])/5
#x轴的单位长度
ymax31 = float("{0:.2f}".format(max(y_list31)))
#y轴的上界
dy31 = ymax31 / 5
#y轴的单位长度
x31 = np.arange(0, xmax31+0.5*dx31, 0.01) #绘直线用
plt.figure(figsize=(16, 10))
plt.scatter(x_list31, y_list311, color='g', marker='+')
plt.plot(x31, r1[0]*x31+r1[1], color='r') #红色
plt.scatter(x_list31, y_list312, color='g', marker='+')
plt.plot(x31, r2[0]*x31+r2[1], color='b') #蓝色
plt.scatter(x_list31, y_list313, color='g', marker='+')
plt.plot(x31, r3[0]*x31+r3[1], color='#924900') #咖啡色
plt.scatter(x_list31, y_list314, color='g', marker='+')
plt.plot(x31, r4[0]*x31+r4[1], color='#FF7700') #橙色
plt.scatter(x_list31, y_list315, color='g', marker='+')
plt.plot(x31, r5[0]*x31+r5[1], color='m') #品红
plt.title('H2.3.1', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(*10^(-3)s)", fontsize=16)
plt.xlim((0, xmax31 + dx31))
plt.ylim((0, ymax31 + dy31))
xaxis = np.arange(0, xmax31 + dx31, dx31)
yaxis = np.arange(0, 2 * ymax31 + 4 * dy31, dy31)
#使数据点落在图像的中央
plt.xticks(xaxis)
plt.yticks(yaxis)
plt.show()

```

数据说明 问题的规模 n ，即列表的长度，范围从 4×10^4 到 1×10^7 ，间隔为 4×10^4 ，每一条 del 操作符执行次，共得到 250 个数据点。设计 5 个函数，分别删除位于列表开头、1/4 长度处、1/2 长度处、3/4 长度处、末尾的 10 个元素。列表足够（元素总个数至少比被删除元素个数高 3 个数量级，最多达到 6 个数量级），保证删除少量元素对问题规模产生影响可以忽略。

运行结果说明

- 斜率: $k_1 = 7.030557596121503$
- 截距: $b_1 = -0.7999599325301242$
- 相关系数: $r_1 = 0.9949504017385908$
- 时间 $t_1(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_1 = 7.03n - 0.80$

- 斜率: $k_2 = 5.147107835645316$
- 截距: $b_2 = -1.3846753349393766$
- 相关系数: $r_2 = 0.9933776455681103$
- 时间 $t_2(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_2 = 5.15n - 1.38$

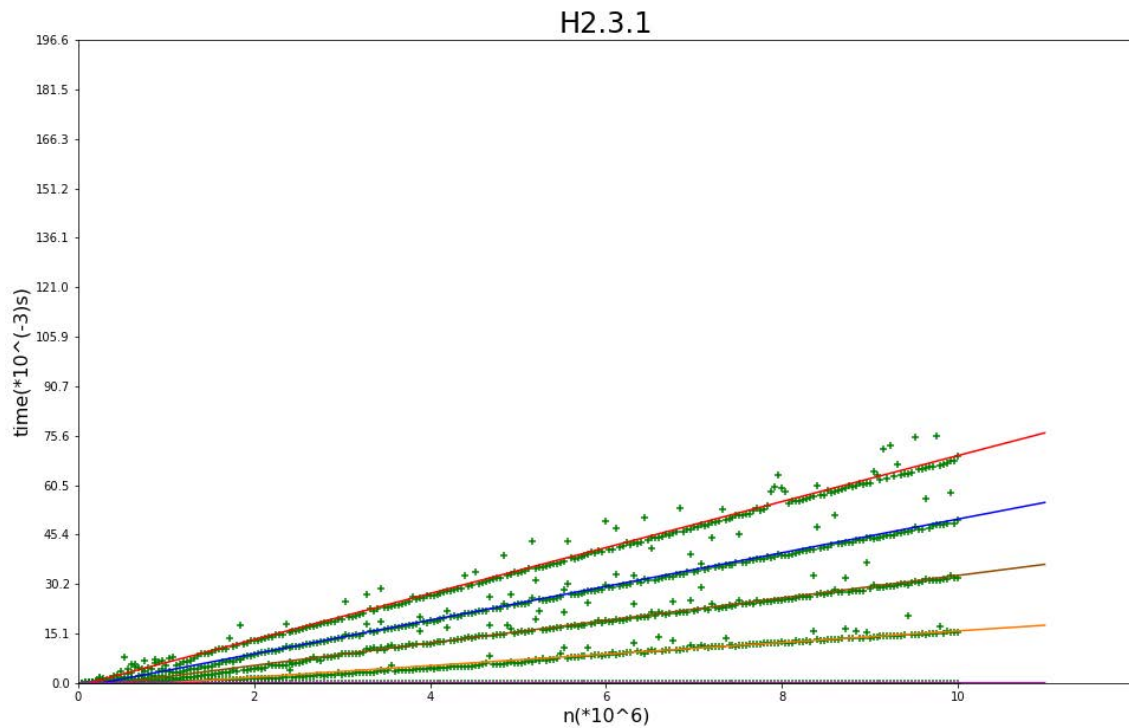
- 斜率: $k_3 = 3.439469756636106$
- 截距: $b_3 = -1.5408877783134176$
- 相关系数: $r_3 = 0.9914362156961903$
- 时间 $t_3(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_3 = 3.44n - 1.54$

- 斜率: $k_4 = 1.7673895809531137$
- 截距: $b_4 = -1.785833696385037$
- 相关系数: $r_4 = 0.9834177637155441$
- 时间 $t_4(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_4 = 7.03n - 0.80$

- 斜率: $k_5 = -3.028560459672879 \times 10^{-5}$
- 截距: $b_5 = 0.0038852337346928765$
- 相关系数: $r_5 = -0.04185167352039433$
- 时间 $t_5(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_5 = 0.00n + 0.00$

实验结果分析 可以看到, 运行 `list` 的 `del` 操作符时, 数据点集中分布在 $t_i = k_i n$ 附近, 说明删除第一个或中间某个元素的时间复杂度为 $O(n)$, 在本次实验中, $t_1 : t_2 : t_3 : t_4 \approx 4 : 3 : 2 : 1$, 与删除点在列表中的位置对应, 而删除最后一个元素时, 数据点集中布在水平线 $t_5 = 0.004 \text{ ms}$ 附近, 拟合出的斜率 k_5 几乎为 0, 说明删除最后一个元素的时间复杂度为 $O(1)$ 。

推测: 这是因为删除列表的第 i 个元素时, 需要将后面 $n - i$ 个元素向前移动一位, 更改元素指标的时间占大多数。导致删除第一个或中某个元素的时间复杂度为 $O(n)$, 而删除最后一个元素不需要移动其它元素的指标, 故时间复杂度为 $O(1)$ 。



3.2 H2.3.2 dict 中 del 操作符的性能

代码

```
# -*- coding: utf-8 -*-
```

```
#[H2.3] 做计时实验，比较list和dict的del操作符性能
```

```
#H2.3.2 dict中del操作符的性能
```

```
import numpy as np
import matplotlib.pyplot as plt
from timeit import Timer

#计时实验程序
def test320():
    for i in range(100000):
        d[n+i] = 0 #插入100000个键值对所需时间
t320=Timer("test320()", "from __main__ import test320")
def test321():
    del d[0]
    d[0] = 0
t321=Timer("test321()", "from __main__ import test321")
def test322():
    del d[int(n/4)]
    d[int(n/4)] = 0
t322=Timer("test322()", "from __main__ import test322")
def test323():
    del d[int(n/2)]
```

```

    d[int(n/2)] = 0
t323=Timer("test323()","from __main__ import test323")
def test324():
    del d[int(3*n/4)]
    d[int(3*n/4)] = 0
t324=Timer("test324()","from __main__ import test324")
def test325():
    del d[n-1]
    d[n-1] = 0
t325=Timer("test325()","from __main__ import test325")
x_list32=[]
y_list321, y_list322 = [], []
y_list323, y_list324, y_list325 = [], [], []
for n in range(10000,2510000,10000): #250 个数据点
    d=dict(list((j,j) for j in range(n)))
    x_list32.append(n)
    y_list321.append(t321.timeit(number=100000)-t320.timeit(number=1))
    y_list322.append(t322.timeit(number=100000)-t320.timeit(number=1))
    y_list323.append(t323.timeit(number=100000)-t320.timeit(number=1))
    y_list324.append(t324.timeit(number=100000)-t320.timeit(number=1))
    y_list325.append(t325.timeit(number=100000)-t320.timeit(number=1))

#线性拟合数据点
def linefit(x, y):
    N = len(x)
    sx,sy,sxx,syy,sxy=0,0,0,0,0
    for i in range(0,N):
        sx += x[i]
        sy += y[i]
        sxx += x[i]*x[i]
        syy += y[i]*y[i]
        sxy += x[i]*y[i]
    k = (sy*sx/N-sxy)/(sx*sx/N-sxx)
    b = (sy-k*sx)/N
    R = (sxy-sx*sy/N)/((sxx-sx*sx/N)*(syy-sy*sy/N))**0.5
    return [k,b,R]
for i in range(len(x_list32)):
    x_list32[i]/=1000000
r1=linefit(x_list32,y_list321)
print('k1='+str(r1[0]))
print('b1='+str(r1[1]))
print('r1='+str(r1[2]))
print('\nt1=%.2fn+%.2f\n'%(r1[0], r1[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)
r2=linefit(x_list32,y_list322)
print('k2='+str(r2[0]))
print('b2='+str(r2[1]))
print('r2='+str(r2[2]))

```

```

print('\nt2=%.2fn+%.2f\n'%(r2[0], r2[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)
r3=linefit(x_list32,y_list323)
print('k3='+str(r3[0]))
print('b3='+str(r3[1]))
print('r3='+str(r3[2]))
print('\nt3=%.2fn+%.2f\n'%(r3[0], r3[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)
r4=linefit(x_list32,y_list324)
print('k4='+str(r4[0]))
print('b4='+str(r4[1]))
print('r4='+str(r4[2]))
print('\nt4=%.2fn+%.2f\n'%(r4[0], r4[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)
r5=linefit(x_list32,y_list325)
print('k5='+str(r5[0]))
print('b5='+str(r5[1]))
print('r5='+str(r5[2]))
print('\nt5=%.2fn+%.2f\n'%(r5[0], r5[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)

#绘图
xmax31 = x_list32[-1]
#x轴的上界
dx31 = (x_list32[-1])/5
#x轴的单位长度
ymax31 = float("{0:.2f}".format(max(y_list321)))
#y轴的上界
dy31 = ymax31 / 5
#y轴的单位长度
x31 = np.arange(0, xmax31+0.5*dx31, 0.01) #绘直线用
plt.figure(figsize=(16, 10))
plt.scatter(x_list32, y_list321, color='g', marker='+')
plt.plot(x31, r1[0]*x31+r1[1], color='r') #红色
plt.scatter(x_list32, y_list322, color='g', marker='+')
plt.plot(x31, r2[0]*x31+r2[1], color='b') #蓝色
plt.scatter(x_list32, y_list323, color='g', marker='+')
plt.plot(x31, r3[0]*x31+r3[1], color='#924900') #咖啡色
plt.scatter(x_list32, y_list324, color='g', marker='+')
plt.plot(x31, r4[0]*x31+r4[1], color='#FF7700') #橙色
plt.scatter(x_list32, y_list325, color='g', marker='+')
plt.plot(x31, r5[0]*x31+r5[1], color='m') #品红
plt.title('H2.3.1', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(s)", fontsize=16)
plt.xlim((0, xmax31 + dx31))
plt.ylim((0, ymax31 + dy31))
xaxis = np.arange(0, xmax31 + dx31, dx31)

```

```

yaxis = np.arange(0, 2 * ymax31 + 4 * dy31, dy31)
#使数据点落在图像的中央
plt.xticks(xaxis)
plt.yticks(yaxis)
plt.show()

```

数据说明 问题的规模 n ，即字典的长度，范围从 1×10^4 到 2.50×10^6 ，间隔为 1×10^4 ，每一条 `del` 操作符和加元素操作符执行 10^6 次，共得到 250 个数据点。设计 5 个函数，分别删除位于字典开头、1/4 长度处、1/2 长度处、3/4 长度处、末的 1 个元素。最后扣除添加 10^6 次元素的时间。

运行结果说明

- 斜率: $k_1 = 0.0007541827139639373$
- 截距: 截距: $b_1 = 0.006889754293975569$
- 关系数: $r_1 = 0.05679696192126313$
- 时间 $t_1(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_1 = 0.00n + 0.01$

- 斜率: $k_2 = 0.00027525989343816105$
- 截距: $b_2 = 0.006621449233735119$
- 相关系数: $r_2 = 0.027747031541544447$
- 时间 $t_2(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_2 = 0.00n + 0.01$

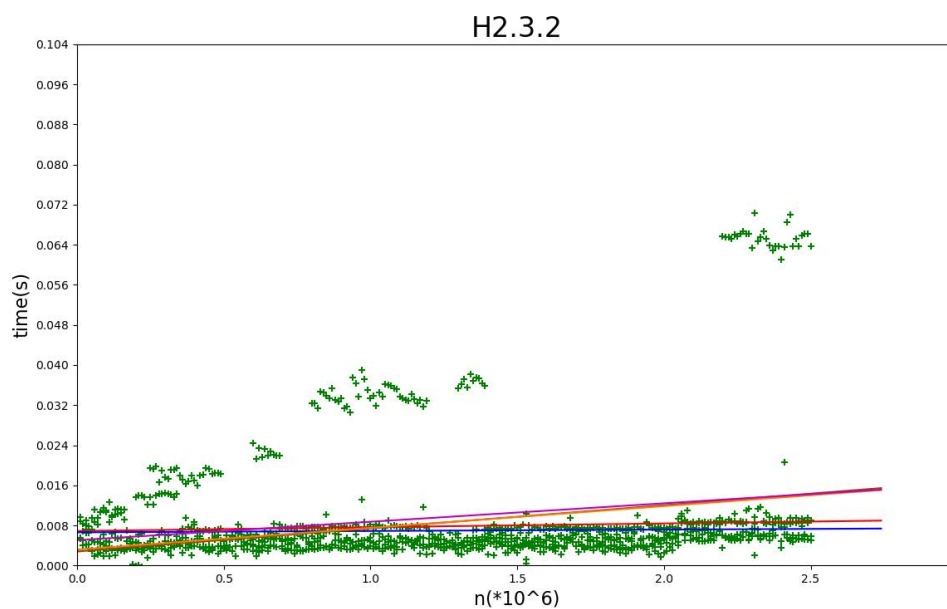
- 斜率: $k_3 = 0.00461939408438505$
- 截距: $b_3 = 0.0027960100240966073$
- 相关系数: $r_3 = 0.2509789046503387$
- 时间 $t_3(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_3 = 0.00n + 0.00$

- 斜率: $k_4 = 0.0043355342811883454$
- 截距: $b_4 = 0.00318401527710871$
- 相关系数: $r_4 = 0.23814047808804087$
- 时间 $t_4(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_4 = 0.00n + 0.00$

- 斜率: $k_5 = 0.0036820635338167445$
- 截距: $b_5 = 0.005051691865060345$
- 相关系数: $r_5 = 0.20667718652716396$
- 时间 $t_5(\text{ms})$ 与问题规模 $n(\times 10^6)$ 的关系: $t_5 = 0.00n + 0.01$

实验结果分析 可以看到, 运行 `dict` 的 `del` 操作符时, 数据点集中分布在水平线 $t_{avg} = 0.005 \text{ ms}$ 附近, 拟合出的斜率 k_i ($i = 1, 2, 3, 4, 5$) 几乎为 0, 说明删除字典中一个元素的时间复杂度为 $O(1)$ 。

推测: 这是因为字典中元素不考虑顺序, 删除一个元素不需要移动其它元素的指标, 故时间复杂度为 $O(1)$ 。



4 H2.4 验证 `list.sort()` 的时间复杂度为 $O(n \log n)$

代码

```
# -*- coding: utf-8 -*-
```

```
#[H2.3] 做计时实验，比较list和dict的del操作符性能
```

```
#H2.3.2 dict中del操作符的性能
```

```
import numpy as np
import matplotlib.pyplot as plt
from timeit import Timer

#计时实验程序
def test320():
    for i in range(100000):
        d[n+i] = 0 #插入100000个键值对所需时间
t320=Timer("test320()", "from __main__ import test320")
def test321():
    del d[0]
    d[0] = 0
t321=Timer("test321()", "from __main__ import test321")
def test322():
    del d[a2]
    d[a2] = 0
t322=Timer("test322()", "from __main__ import test322")
def test323():
    del d[a3]
    d[a3] = 0
t323=Timer("test323()", "from __main__ import test323")
def test324():
    del d[a4]
    d[a4] = 0
t324=Timer("test324()", "from __main__ import test324")
def test325():
    del d[a5]
    d[a5] = 0
t325=Timer("test325()", "from __main__ import test325")
x_list32=[]
y_list321, y_list322 = [], []
y_list323, y_list324, y_list325 = [], [], []
for n in range(10000,251000,10000): #250个数据点
    a2=int(n/4)
    a3=int(n/2)
    a4=int(3*n/4)
    a5=n-1
    d=dict(list((j,j) for j in range(n)))
    x_list32.append(n)
```

```

y_list321.append(t321.timeit(number=100000)-t320.timeit(number=1))
y_list322.append(t322.timeit(number=100000)-t320.timeit(number=1))
y_list323.append(t323.timeit(number=100000)-t320.timeit(number=1))
y_list324.append(t324.timeit(number=100000)-t320.timeit(number=1))
y_list325.append(t325.timeit(number=100000)-t320.timeit(number=1))

#线性拟合数据点
def linefit(x, y):
    N = len(x)
    sx,sy,sxx,syy,sxy=0,0,0,0,0
    for i in range(0,N):
        sx += x[i]
        sy += y[i]
        sxx += x[i]*x[i]
        syy += y[i]*y[i]
        sxy += x[i]*y[i]
    k = (sy*sx/N-sxy)/(sx*sx/N-sxx)
    b = (sy-k*sx)/N
    R = (sxy-sx*sy/N)/((sxx-sx*sx/N)*(syy-sy*sy/N))**0.5
    return [k,b,R]
for i in range(len(x_list32)):
    x_list32[i]/=1000000
r1=linefit(x_list32,y_list321)
print('k1='+str(r1[0]))
print('b1='+str(r1[1]))
print('r1='+str(r1[2]))
print('\nt1=%.2fn+%.2f\n'%(r1[0], r1[1]))
#拟合结果（斜率k，截距b，相关系数r）
r2=linefit(x_list32,y_list322)
print('k2='+str(r2[0]))
print('b2='+str(r2[1]))
print('r2='+str(r2[2]))
print('\nt2=%.2fn+%.2f\n'%(r2[0], r2[1]))
#拟合结果（斜率k，截距b，相关系数r）
r3=linefit(x_list32,y_list323)
print('k3='+str(r3[0]))
print('b3='+str(r3[1]))
print('r3='+str(r3[2]))
print('\nt3=%.2fn+%.2f\n'%(r3[0], r3[1]))
#拟合结果（斜率k，截距b，相关系数r）
r4=linefit(x_list32,y_list324)
print('k4='+str(r4[0]))
print('b4='+str(r4[1]))
print('r4='+str(r4[2]))
print('\nt4=%.2fn+%.2f\n'%(r4[0], r4[1]))
#拟合结果（斜率k，截距b，相关系数r）
r5=linefit(x_list32,y_list325)
print('k5='+str(r5[0]))

```

```

print('b5='+str(r5[1]))
print('r5='+str(r5[2]))
print('\nt5=%.2fn+%.2f\n'%(r5[0], r5[1]))
#拟合结果 (斜率k, 截距b, 相关系数r)

#绘图
xmax31 = x_list32[-1]
#x轴的上界
dx31 = (x_list32[-1])/5
#x轴的单位长度
ymax31 = float("{0:.2f}".format(max(y_list321)))
#y轴的上界
dy31 = ymax31 / 5
#y轴的单位长度
x31 = np.arange(0, xmax31+0.5*dx31, 0.01) #绘直线用
plt.figure(figsize=(16, 10))
plt.scatter(x_list32, y_list321, color='g', marker='+')
plt.plot(x31, r1[0]*x31+r1[1], color='r') #红色
plt.scatter(x_list32, y_list322, color='g', marker='+')
plt.plot(x31, r2[0]*x31+r2[1], color='b') #蓝色
plt.scatter(x_list32, y_list323, color='g', marker='+')
plt.plot(x31, r3[0]*x31+r3[1], color='#924900') #咖啡色
plt.scatter(x_list32, y_list324, color='g', marker='+')
plt.plot(x31, r4[0]*x31+r4[1], color='#FF7700') #橙色
plt.scatter(x_list32, y_list325, color='g', marker='+')
plt.plot(x31, r5[0]*x31+r5[1], color='m') #品红
plt.title('H2.3.2', fontsize=24)
plt.xlabel("n(*10^6)", fontsize=16)
plt.ylabel("time(s)", fontsize=16)
plt.xlim((0, xmax31 + dx31))
plt.ylim((0, ymax31 + dy31))
xaxis = np.arange(0, xmax31 + dx31, dx31)
yaxis = np.arange(0, 2 * ymax31 + 4 * dy31, dy31)
#使数据点落在图像的中央
plt.xticks(xaxis)
plt.yticks(yaxis)
plt.show()

```

数据说明 问题的规模 n ，即列表 `l` 的长度，范围从 1×10^5 到 1×10^6 ，间隔为 4×10^3 ，每一条按索引取值的程序指令重复行 1 次，共得到 250 个数据点。

运行结果说明

- 斜率: $k = 2.6344297060965576$
- 截距: $b = -0.02142297511362065$
- 相关系数: $r = 0.9992198897036473$

- 时间 $t(s)$ 与问题规模的对数 $\log_2(n)(\times 10^8)$ 的关系: $t = 2.63 n \log_2(n) - 0.02$

实验结果分析 可以看到, 运行 `list.sort()` 函数 1 次时, 所用时间 t 与问题规模 (即列表长度 n) 的对数线性 $n \log n$ 之间呈线性关系, 性相关系数 r 高达 0.999 (线性相关性非常高, 线性回归方程有意义), 说明 `list.sort()` 函数的时间复杂度为 $O(n \log n)$

